

Project Brief: Risiko- og avhengighetsoversikt for prosjektledere

1. Bakgrunn og problem

Prosjektledere har ofte risikoer og avhengigheter spredt på flere steder: Excel-ark, møtereferater, statusrapporter, e-poster og uformelle avklaringer. Dette gjør det vanskelig å få en samlet og oppdatert oversikt over hva som faktisk kan påvirke fremdrift, kostnad, kvalitet eller beslutningsevne.

Typiske utfordringer:

- Risikoer registreres ulikt fra prosjekt til prosjekt.
- Kritiske forhold drukner i lange dokumenter eller ustrukturerte møtereferater.
- Det er uklart hvem som eier risikoer og avhengigheter.
- Tiltak og status følges ikke alltid systematisk opp.
- Avhengigheter mellom leveranser, beslutninger og eksterne aktører blir ofte synlige for sent.
- Det er vanskelig å se sammenheng mellom risikoer og avhengigheter.

Formålet med prototypen er å vise hvordan et enkelt, lokalt webverktøy kan samle risikoer og avhengigheter i én oversiktlig løsning.

2. Mål

Første versjon skal være en lokal webprototype som demonstrerer konseptet, ikke et produksjonssystem.

Mål for prototypen:

- Gi prosjektleder en samlet oversikt over risikoer.
- Vise risikoer i en enkel risikomatrise basert på sannsynlighet og konsekvens.
- Gi oversikt over avhengigheter som kan påvirke fremdrift.
- Synliggjøre eierskap, status og tiltak.
- Gjøre det mulig å koble risikoer og avhengigheter der det er relevant.
- Bruke eksempeldata slik at prototypen fungerer som demo uten oppsett.
- Lagre data lokalt i nettleseren med `localStorage`.

3. Målgruppe

Primær målgruppe:

- Prosjektledere
- Delprosjektledere
- Prosjektstøtte / PMO
- Leveranseansvarlige

Sekundær målgruppe:

- Styringsgrupper
- Produkteiere
- Teamledere
- Konsulenter som bistår med prosjektstyring

4. Avgrensninger

Første versjon skal være enkel og lokal.

Innenfor scope:

- React-basert webprototype.
- TypeScript.
- Vite.
- Lokal lagring med `localStorage`.
- Eksempeldata ved første oppstart.
- Enkel CRUD-funksjonalitet for risikoer og avhengigheter.
- Filtrering på status, ansvarlig og kritikalitet.
- Enkel kobling mellom risikoer og avhengigheter.
- Responsivt nok til desktop-demo.

Utenfor scope:

- Innlogging og tilgangsstyring.
- Backend/API.
- Database.
- Integrasjoner mot Excel, SharePoint, Jira, Teams eller andre systemer.
- Sanntidssamarbeid.
- Varslinger.
- Kompleks rapportering.
- Rollemodell.
- Import/eksport i første versjon.
- Historikk og revisjonsspor.
- Produksjonsklar sikkerhet.

5. Overordnet løsningsidé

Løsningen skal være en enkel single-page application der prosjektleder kan se og vedlikeholde risikoer og avhengigheter.

Foreslåtte hovedvisninger:

1. Dashboard

Kort oppsummering av antall åpne risikoer, kritiske risikoer, åpne avhengigheter og forsinkede/ uklare avklaringer.

2. Risikooversikt

Tabell eller kortvisning med risikoer, sannsynlighet, konsekvens, score, status, ansvarlig og tiltak.

3. Risikomatrise

5x5-matrise der risikoer plasseres etter sannsynlighet og konsekvens.

4. Avhengigheter

Liste over avhengigheter mot leveranser, beslutninger, eksterne aktører eller andre prosjekter.

5. Detalj-/redigeringspanel

Enkel visning for å opprette og redigere risikoer og avhengigheter.

6. Funksjonelle behov

6.1 Risikoer

Brukeren skal kunne:

- Se en liste over risikoer.
- Opprette ny risiko.
- Redigere eksisterende risiko.
- Slette risiko.
- Registrere sannsynlighet fra 1 til 5.
- Registrere konsekvens fra 1 til 5.
- Se beregnet risikoscore.
- Registrere ansvarlig person.
- Registrere status.
- Registrere tiltak.
- Registrere frist for tiltak eller oppfølging.
- Koble risiko til én eller flere avhengigheter.
- Filtrere risikoer på status, ansvarlig og risikonivå.

6.2 Risikomatrise

Brukeren skal kunne:

- Se risikoer plassert i en 5x5-matrise.
- Skille mellom lav, middels og høy risiko basert på score.
- Klikke på en risiko i matrisen og se detaljer.
- Se hvor mange risikoer som finnes i hver celle.

6.3 Avhengigheter

Brukeren skal kunne:

- Se en liste over avhengigheter.
- Opprette ny avhengighet.
- Redigere eksisterende avhengighet.
- Slette avhengighet.
- Registrere type avhengighet.
- Registrere eier/ansvarlig.
- Registrere status.
- Registrere frist eller forventet avklaringsdato.
- Koble avhengighet til én eller flere risikoer.

Mulige typer avhengigheter:

- Intern leveranse
- Ekstern aktør
- Beslutning
- Teknisk avklaring
- Ressurstilgang
- Juridisk/anskaffelse
- Annet

6.4 Filtrering og sortering

Brukeren skal kunne filtrere på:

- Status
- Ansvarlig
- Risikonivå
- Type avhengighet

Brukeren bør kunne sortere risikoer på:

- Score
- Frist
- Status
- Ansvarlig

7. Ikke-funksjonelle behov

- Løsningen skal kunne kjøres lokalt med `npm install` og `npm run dev`.
- Løsningen skal ikke kreve backend eller miljøvariabler.
- Data skal lagres i `localStorage`.
- Ved første oppstart skal eksempeldata lastes inn.
- Brukeren skal kunne nullstille demo-data.
- Koden skal være enkel å forstå og videreutvikle.
- UI-et skal være ryddig og egnet for demonstrasjon i nettleser.

8. Teknisk rammeverk

Antatt teknologistack:

- React
- TypeScript
- Vite
- localStorage
- CSS Modules, vanlig CSS eller enkel komponentbasert styling

Foreslått prosjektstruktur:

```
src/  
  components/
```

```
DashboardSummary.tsx
RiskList.tsx
RiskMatrix.tsx
RiskForm.tsx
DependencyList.tsx
DependencyForm.tsx
Filters.tsx
data/
  seedData.ts
hooks/
  useLocalStorage.ts
models/
  risk.ts
  dependency.ts
utils/
  riskScoring.ts
App.tsx
main.tsx
```

9. Enkel løsningsarkitektur

Prototypen kan bygges som en ren frontend-applikasjon.

Arkitektur:

```
React UI
|
|-- State management i App eller custom hooks
|
|-- localStorage adapter
|
|-- Seed data ved første oppstart
|
|-- Presentasjonskomponenter for lister, matrise og skjema
```

Dataflyt:

1. Applikasjonen starter.
2. Den sjekker om det finnes lagrede data i `localStorage`.
3. Hvis ikke, lastes eksempeldata fra `seedData.ts`.
4. Brukeren oppretter, endrer eller sletter risikoer og avhengigheter.
5. Endringer lagres automatisk til `localStorage`.
6. UI oppdateres basert på gjeldende state og aktive filtre.

10. Datamodell

10.1 Risk

```
export type RiskStatus = "open" | "inProgress" | "mitigated" | "closed";

export interface Risk {
  id: string;
  title: string;
  description: string;
  probability: 1 | 2 | 3 | 4 | 5;
  impact: 1 | 2 | 3 | 4 | 5;
  owner: string;
  status: RiskStatus;
  mitigation: string;
  dueDate?: string;
  dependencyIds: string[];
  createdAt: string;
  updatedAt: string;
}
```

Beregnet felt:

```
riskScore = probability * impact;
```

Forslag til risikonivå:

```
1-5    = low
6-12   = medium
15-25  = high
```

10.2 Dependency

```
export type DependencyStatus = "notStarted" | "inProgress" | "blocked" |
"resolved";

export type DependencyType =
  | "internalDelivery"
  | "externalParty"
  | "decision"
  | "technicalClarification"
  | "resource"
  | "procurement"
  | "other";

export interface Dependency {
  id: string;
```

```

title: string;
description: string;
type: DependencyType;
owner: string;
status: DependencyStatus;
dueDate?: string;
relatedRiskIds: string[];
createdAt: string;
updatedAt: string;
}

```

10.3 Filter state

```

export interface Filters {
  owner?: string;
  riskStatus?: RiskStatus | "all";
  dependencyStatus?: DependencyStatus | "all";
  riskLevel?: "low" | "medium" | "high" | "all";
  dependencyType?: DependencyType | "all";
}

```

11. Eksempeldata

Prototypen bør ha eksempeldata som viser realistiske prosjektutfordringer.

Eksempler på risikoer:

1. Forsinket beslutning om løsningsvalg
 Sannsynlighet: 4
 Konsekvens: 5
 Ansvarlig: Prosjektleder
 Tiltak: Eskalere beslutningsbehov til styringsgruppen.
2. Manglende tilgang til nøkkelressurser
 Sannsynlighet: 3
 Konsekvens: 4
 Ansvarlig: Ressursansvarlig
 Tiltak: Avklare ressursplan med linjeleder.
3. Uklart om ekstern leverandør kan levere innen frist
 Sannsynlighet: 3
 Konsekvens: 5
 Ansvarlig: Leveranseansvarlig
 Tiltak: Etablere ukentlig oppfølging med leverandør.

Eksempler på avhengigheter:

1. Beslutning om målarkitektur
Type: Beslutning
Status: Ikke startet
Ansvarlig: Styringsgruppe
2. Leveranse fra ekstern integrasjonspartner
Type: Ekstern aktør
Status: Pågår
Ansvarlig: Integrasjonsansvarlig
3. Tilgang til testmiljø
Type: Teknisk avklaring
Status: Blokkert
Ansvarlig: Teknisk leder

12. Brukerflyt

12.1 Se samlet status

1. Brukeren åpner prototypen.
2. Dashboard viser nøkkeltall:
3. Antall åpne risikoer.
4. Antall høy risiko.
5. Antall åpne avhengigheter.
6. Antall blokkerte avhengigheter.
7. Brukeren kan klikke seg videre til risikoer eller avhengigheter.

12.2 Følge opp risiko

1. Brukeren åpner risikooversikten.
2. Brukeren filtrerer på status eller ansvarlig.
3. Brukeren velger en risiko.
4. Brukeren oppdaterer tiltak, ansvarlig, status eller frist.
5. Endringen lagres lokalt.

12.3 Se kritiske risikoer i matrise

1. Brukeren åpner risikomatrisen.
2. Risikoer vises basert på sannsynlighet og konsekvens.
3. Brukeren klikker på en risiko i høy-risiko-området.
4. Detaljer vises i sidepanel eller modal.

12.4 Følge opp avhengighet

1. Brukeren åpner avhengighetslisten.
2. Brukeren filtrerer på status eller type.

3. Brukeren åpner en avhengighet.
4. Brukeren ser hvilke risikoer som er koblet til avhengigheten.
5. Brukeren oppdaterer status eller ansvarlig.

13. Foreslåtte epics og issues

Epic 1: Prosjektoppsett

Formål: Etablere teknisk grunnstruktur.

Issues:

- Sett opp Vite med React og TypeScript.
- Opprett grunnleggende prosjektstruktur.
- Legg til enkel global styling.
- Opprett TypeScript-modeller for risikoer og avhengigheter.
- Lag seed-data for demo.

Akseptansekriterier:

- Appen starter med `npm run dev`.
- TypeScript kompilerer uten feil.
- Seed-data er tilgjengelig i applikasjonen.

Epic 2: Lokal datalagring

Formål: Gjøre prototypen brukbar uten backend.

Issues:

- Lag `useLocalStorage` hook.
- Last seed-data ved første oppstart.
- Lagre endringer automatisk i `localStorage`.
- Legg til funksjon for å nullstille demo-data.

Akseptansekriterier:

- Data bevares etter refresh.
- Første oppstart viser eksempeldata.
- Brukeren kan nullstille data til opprinnelig demo-oppsett.

Epic 3: Risikooversikt

Formål: Gi brukeren oversikt over registrerte risikoer.

Issues:

- Lag `RiskList` komponent.
- Vis tittel, eier, status, sannsynlighet, konsekvens og score.
- Lag enkel opprett/rediger-funksjon.
- Lag slett-funksjon.

- Vis risikonivå basert på score.

Akseptansekriterier:

- Brukeren kan se alle risikoer.
- Brukeren kan opprette, redigere og slette risikoer.
- Score beregnes automatisk.
- Risikoer kategoriseres som lav, middels eller høy.

Epic 4: Risikomatrise

Formål: Visualisere risikoer etter sannsynlighet og konsekvens.

Issues:

- Lag 5x5 risikomatrise.
- Plasser risikoer i riktig celle.
- Vis antall risikoer per celle.
- Gjør risikoer klikkbare.
- Vis detaljer for valgt risiko.

Akseptansekriterier:

- Alle risikoer vises i riktig matriseplassering.
- Brukeren kan identifisere høyrisiko-områder.
- Klikk på risiko viser relevant informasjon.

Epic 5: Avhengighetsoversikt

Formål: Gi strukturert oversikt over prosjektavhengigheter.

Issues:

- Lag `DependencyList` komponent.
- Vis tittel, type, eier, status og frist.
- Lag opprett/rediger-funksjon.
- Lag slett-funksjon.
- Vis koblede risikoer.

Akseptansekriterier:

- Brukeren kan se alle avhengigheter.
- Brukeren kan opprette, redigere og slette avhengigheter.
- Koblede risikoer vises i avhengighetsdetaljer.

Epic 6: Kobling mellom risikoer og avhengigheter

Formål: Synliggjøre sammenhenger mellom risikoer og avhengigheter.

Issues:

- Gjør det mulig å velge avhengigheter i risikoskjema.

- Gjør det mulig å vise risikoer knyttet til en avhengighet.
- Hold relasjoner konsistente ved redigering og sletting.

Akseptansekriterier:

- En risiko kan kobles til én eller flere avhengigheter.
- En avhengighet kan vise relaterte risikoer.
- Sletting av risiko eller avhengighet håndteres uten krasj.

Epic 7: Filtrering og demo-egnet UI

Formål: Gjøre prototypen enkel å demonstrere.

Issues:

- Lag filtre for status.
- Lag filtre for ansvarlig.
- Lag filtre for risikonivå.
- Lag filtre for avhengighetstype.
- Lag dashboard med nøkkeltall.
- Forbedre layout for desktop-demo.

Akseptansekriterier:

- Filtre påvirker visningen uten reload.
- Dashboard viser oppdaterte nøkkeltall.
- UI-et er ryddig nok til å brukes i en demo.

14. Akseptansekriterier for første prototype

Prototypen anses som vellykket når:

- Den kan kjøres lokalt uten backend.
- Den viser eksempeldata ved første oppstart.
- Brukeren kan se en risikooversikt.
- Brukeren kan se en risikomatrise.
- Brukeren kan se en avhengighetsoversikt.
- Brukeren kan opprette og redigere risikoer.
- Brukeren kan opprette og redigere avhengigheter.
- Risikoer kan kobles til avhengigheter.
- Data lagres i `localStorage`.
- Brukeren kan filtrere på status og ansvarlig.
- UI-et er enkelt nok til å forklare konseptet på under fem minutter.

15. Forslag til første vertikale slice

Første vertikale slice bør være liten, men demonstrere hele verdikjeden fra data til visning og oppdatering.

Scope for første slice

Bygg følgende:

1. App-shell med enkel navigasjon.
2. Seed-data for risikoer og avhengigheter.
3. Lokal lagring i `localStorage`.
4. Risikooversikt med listevissning.
5. Risikomatrise basert på risikoenes sannsynlighet og konsekvens.
6. Enkel redigering av risiko.
7. Avhengighetsliste som viser koblede risikoer.

Ikke ta med i første slice

- Full CRUD for alle entiteter.
- Avansert filtrering.
- Modaldesign eller avansert styling.
- Import/eksport.
- Brukerroller.
- Backend.

Akseptansekriterier for første slice

- Appen starter lokalt med eksempeldata.
- Risikoer vises både i liste og matrise.
- Endring av sannsynlighet eller konsekvens oppdaterer matrisen.
- Endringer lagres etter refresh.
- Minst én risiko er koblet til en avhengighet.
- Brukeren kan forklare konseptet basert på skjermbildene uten å måtte vise kode.

16. Prioritert backlog for MVP-demo

Prioritet 1:

- Prosjektoppsett med React, TypeScript og Vite.
- Datamodeller for risiko og avhengighet.
- Seed-data.
- `localStorage`-lagring.
- Risikooversikt.
- Risikomatrise.

Prioritet 2:

- Avhengighetsoversikt.
- Kobling mellom risikoer og avhengigheter.
- Redigering av risikoer.
- Redigering av avhengigheter.

Prioritet 3:

- Filtre.
- Dashboard.

- Nullstill demo-data.
- Bedre styling og layout.

17. Designprinsipper

- Oversikt først, detaljer etterpå.
- Vis kritiske forhold tydelig.
- Ikke skjul eierskap.
- Gjør status lett å oppdatere.
- Hold skjemaene korte.
- Ikke bygg mer system enn nødvendig for demo.

18. Mulige videreutvidelser

Ikke del av første versjon, men aktuelle senere:

- Eksport til Excel eller CSV.
- Import fra Excel.
- Enkel rapportvisning.
- Kommentarer på risikoer og avhengigheter.
- Endringshistorikk.
- Varsling om frister.
- Integrasjon med prosjektverktøy.
- Deling mellom brukere.
- Backend og database.
- Autentisering og tilgangsstyring.
- AI-basert forslag til risikoer basert på møtereferater eller statusrapporter.

19. Suksesskriterier for demo

Demoen bør vise at:

- Prosjektleder raskt kan se hva som er kritisk.
- Risikoer og avhengigheter kan samles i ett enkelt bilde.
- Eierskap og tiltak blir tydeligere.
- Sammenhenger mellom risikoer og avhengigheter blir synlige.
- Konseptet kan videreutvikles uten å starte med et tungt system.